Ch 10: Object-Oriented Programming (OOP) and Classes

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Procedural Programming vs. Object-Oriented Programming (OOP)

Procedural vs. OOP

Procedural programming is a method of writing software. It is a programming practice centered on the procedures (functions) that take place in a program. Object-oriented programming is centered on objects. Objects are created from abstract data types that encapsulate data and functions together.

Procedural Programming

- **Procedural programming**: writing programs made of functions that perform specific tasks
  - Procedures typically operate on data items that are separate from the procedures
  - Data items passed from one procedure to another
  - Focus: to create procedures that operate on data

Object-oriented programming

- **Object-oriented programming**: focus creating objects
- **Object**: entity that contains data **AND** procedures
  - Data is known as data attributes, member variables, etc
  - Procedures are known as member methods
    - Methods perform operations on data attributes of object



Object-Oriented Programming

- **Encapsulation**: combining data and code into a single object
- **Data hiding**: object's data attributes are hidden from code outside the object
  - Access restricted to the object's methods
    - Protects from accidental corruption
    - Outside code does not need to know internal structure of the object
- **Object reusability**: the same object can be used in different programs
  - Example: 3D image object can be used for architecture and game programming

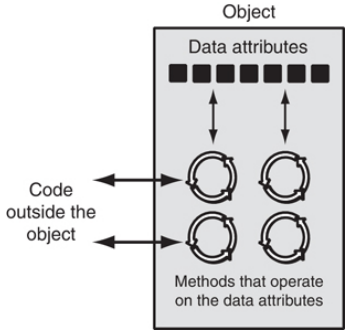


Figure 1: An object encapsulates data attributes and methods that act on the objects data. Code outside interacts with object's methods, data attributes are hidden. (Gaddis, 2021, pg.545)

EAST TEXAS A&M

An Everyday Example of an Object

- **Data attributes**: define the state of an object
  - Example: a clock object would have `second`, `minute` and `hour` data attributes
  - all integers, though second and minute should be restricted to 00-59 and hours to 00-23 (if using military time representation)
- **Public methods**: allow external code to manipulate the object
  - Example: `set_time()` takes second, minute and hour as input and sets the time, checking that attributes are valid
  - Example: `set_alarm_time()` if implementing an alarm clock, have a second set of attributes for when an alarm should sound, and set them
- **Private methods**: used for object's inner workings
  - Example: methods to update the time every second or to check and sound the alarm. These should not be callable from outside.



EAST TEXAS A&M

Classes

Classes

A class is code that specifies the data attributes and methods for a particular type of object.

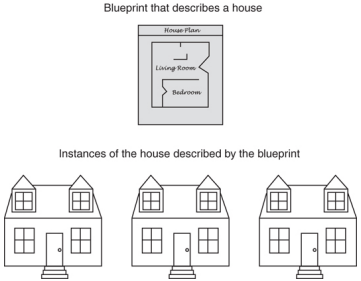


Figure 2: A blueprint and houses built from the blueprint (Gaddis, 2021, pg.547)

- **Class**: code that specifies the data attributes and methods of a particular type of object
  - Similar to a blueprint of a house or a cookie cutter
- **Instance**: an object created from a class
  - Similar to a specific house built according to the blueprint or a specific cookie cut from the cookie cutter
  - There can be many instances of one class

EAST TEXAS A&M

Class Definitions (1 of 2)

- **Class definition**: set of statements that define a class's methods and data attributes
  - Format `class ClassName`
  - Class names conventionally use SnakeCase with initial upper case letter for class name
- Method definition like any other Python function, but with additional 1st parameter
  - `self` parameter: required in every method in the class, references specific object that method is working on
- **Initializer method**: automatically executed when an instance of the class is created
  - Initializes the object's data attributes and assigns `self` parameter to the object that was created
  - Format: `def __init__(self):`
  - Usually the first method in a class definition

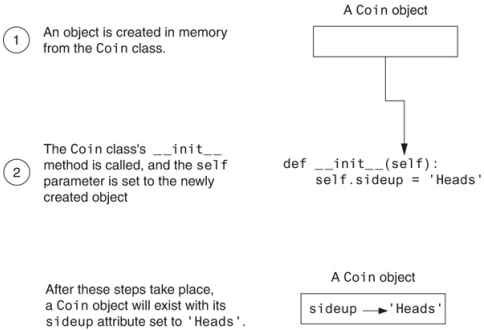


Figure 3: Actions caused when Coin object created (Gaddis, 2021, pg.553)

EAST TEXAS A&M

Class Definitions (2 of 2)

- To create a new instance of a class call the initializer method
  - Format: `my_instance = ClassName()`
- To call any of the class methods using the created instance, use dot notation
  - Format: `my_instance.method()`
  - Because the `self` parameter references the specific instance of the object, the method will affect this instance
    - Reference to `self` is passed automatically

```
# see coin_demo1.py
my_coin = Coin()
my_coin.toss()
```



Hiding Attributes and Storing Classes in Modules

An object's data attributes should be private

- To make sure of this, place two underscores __ in front of attribute name
- Example: `__current_minute`

Classes can be stored in modules

- Filename for module must end in `.py`
- Module can be imported to programs that use the class
- This is no different from using regular functions imported from a module that you have seen before
- See `bankaccount.py` example



EAST TEXAS A&M

The `__str__` Method

- **Object's state**: the values of the object's attribute at a given moment
- `__str__` method: displays the object's state (by convention)
  - Automatically called when the object is passed as an argument to the `print()` function
  - Automatically called when the object is passed as an argument to the `str()` function
- There are other methods that have similar special meaning in Python OOP
  - Always begin and end with two underscores `__special_method__`
  - See Python Standard Library Language Reference, 3.3 Special method names for others (Python, 2026)

EAST TEXAS A&M

Summary

- Object-oriented programming (OOP) focuses on defining and creating user defined objects
  - An object encapsulates data and code
  - An object hides private attributes from outside direct access
- A **class** defines a blueprint or template to create objects
  - You can create multiple instances of an object from the class definition, by invoking the initializer method
  - Hidden attributes should be declared by using two underscores for the attribute name, e.g. `__current_minute`
  - The `__str__` method is used to display objects state by `print()`,
  - In general other methods of the form `__special_method__` can be defined to do particular things with Python classes



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

Python. (2026). *Python 3 standard library documentation*. Retrieved August 13, 2026, from https://docs.python.org/3/

EAST TEXAS A&M